FIT2179

Data Visualisation

Topic 1

Introduction to Visualisation - What, Why, How

Course Notes

Release Date February 28, 2026

Prepared By Songhai Fan and Bernhard Jenny, Monash University

Introduction to Visualisation - What, Why, How

IN THIS TOPIC

- We establish core definitions of data visualisation and clarify the difference between representation and presentation.
- We explain why visualisation matters by focusing on human judgement, visual perception, and external memory.
- We show why summary statistics are not enough by using Anscombe's Quartet and the Datasaurus example.
- We introduce Munzner's four-level framework and connect it to the three design questions: what, why, and how.
- We close with a concrete walkthrough of the what-why-how method using the Gapminder bubble chart.

1.1 Defining Data Visualisation

There is **no single, universally accepted definition** of data visualisation. A widely used working definition describes it as *"the visual representation and presentation of data to facilitate understanding"* (Kirk 2019). Here, **representation** refers to the transformation of data into visual marks and chart structures, whereas **presentation** concerns how those representations are situated through interaction, layout, annotation, and interface context.

Kirk's phrase "facilitate understanding" can be read as a three-step progression (Kirk 2019, p. 29). First, viewers **perceive**: they notice what is visible in the chart. Next, they **interpret**: they connect those visible elements to context. Finally, they **comprehend**: they decide what the information means for their own question or action.

To make this boundary clearer, Kirk also separates data visualisation from several adjacent terms (Kirk 2019, pp. 27–29):

SOURCES
Definitions

Kirk 2019
Data Visualisation: A Handbook for Data Driven Design

Munzner 2014
Visualization Analysis and Design

- **Infographics** usually focus on explanatory communication and often combine charts with illustration and text.
- **Information visualisation** is often used for abstract structures and relationships, such as trees and networks.
- **Information design** is a broader practice that includes many functional communication artefacts beyond charts.
- **Data art** primarily emphasises expression and aesthetics rather than analytical understanding.
- **Dashboards** are multi-view compositions, so they should not be treated as a single chart type.
- **Storytelling** requires narrative context; a chart alone does not automatically form a story.

1.2 Why Visualisation Matters

After clarifying what visualisation is, the next question is why we still need it when advanced algorithms are available. The key point is simple: visualisation does not replace computation. It complements computation by keeping human judgement in the analytical loop.

Munzner argued that *"Computer-based visualization systems provide visual representations of datasets designed to help people carry out tasks more effectively."* (Munzner 2014). This means that visualisation is suitable when there is a need to augment human capabilities rather than replace people with computational decision-making methods.

The human in the loop

Visualisation is especially valuable when tasks are ill-specified. In many real settings, analysts do not know the exact question in advance, and the question itself evolves as evidence appears. In this situation, fully automated pipelines are limited because they require precise objectives early. Visualisation supports iterative sensemaking by allowing people to form, test, and revise hypotheses while directly inspecting the data.

NOTE
Generally it's better to have an image than a number. Humans are not good at interpreting numbers - Better to combine multi-dimensional information into a single, easily understandable form - Easier to extract and emphasise important info.

Advantages of Visual Perception

The visual system provides a high-bandwidth channel to the brain, so people can process many signals in parallel. In practice, this advantage comes from speed (gist recognition in about 20 milliseconds), parallel processing, and **pre-attentive** detection. Visualisations also act as **external memory**: analysts can offload details to a visual artefact and return later to compare patterns and hypotheses.

Where visualisation is used in practice

Visualisation supports more than one task stage. In task terms, we often describe this as **discover–design–communicate**, which is a different view of the same core pipeline used in this topic. A public-transport planning case fits this pattern: inspect accessibility patterns first, test design options second, and present the chosen network change to decision makers third.

Visualisation sits at the intersection of multiple fields: **knowledge discovery**, **cognitive science**, **graphic design**, **interactive computer graphics**, and **data science**. In this unit, we focus on selected ideas from each field rather than treating visualisation as chart styling alone.

1.3 Why Show the Data, Not Just the Summary?

SOURCES
Anscombe's Quartet
Anscombe 1973
"Graphs in Statistical Analysis"
Matejka and Fitzmaurice 2017
"Same Stats, Different Graphs: Generating Datasets with Varied Appearance and Identical Statistics through Simulated Annealing"
Open source link

The value of detailed visual inspection is clear in **Anscombe's Quartet**. It contains four datasets with identical summary statistics. The means, variances, correlations, and regression lines match, but the plotted structures are very different. Visualisation exposes outliers, nonlinearity, and unusual patterns that summary values hide.

One useful refinement is to compare Anscombe's Quartet with an *unstructured quartet* (random-looking point clouds matched on the same summary statistics). Datasets can share summary values without being clearly distinct by eye. This is why Anscombe's original construction is so effective for teaching: the panels are statistically matched but visually different in meaningful ways.

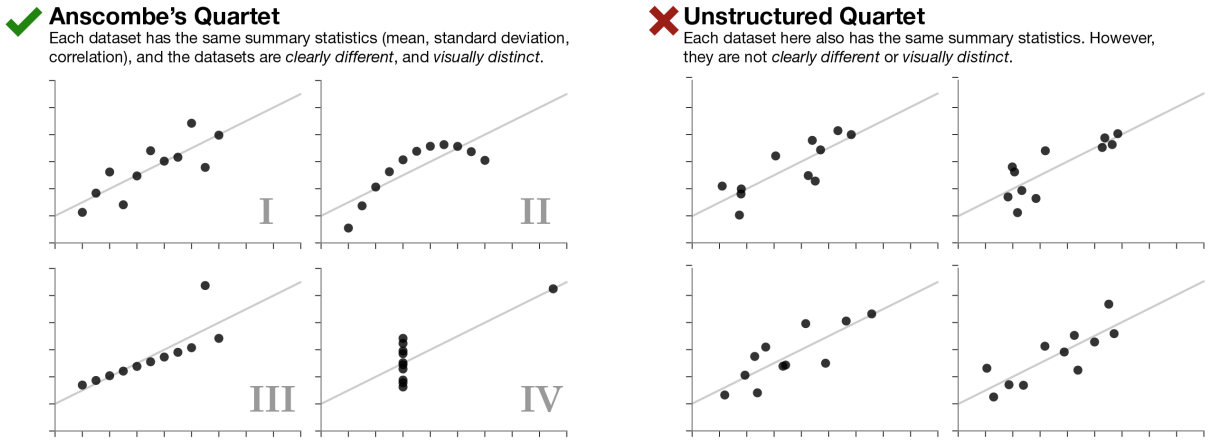

Figure 1.1: Anscombe's Quartet (left) compared with an unstructured quartet (right): both can share summary statistics, but only the left side is clearly visually distinct (Anscombe 1973; Matejka and Fitzmaurice 2017).

Figure 1.1 makes the mismatch explicit. Dataset I is roughly linear with moderate noise, Dataset II shows curvature, Dataset III is dominated by an outlier, and Dataset IV concentrates points at one x-value with a high-

leverage point. The same summary statistic values therefore do not imply the same analytical story.

This lesson generalises directly: *important structure can be hidden in summary metrics*. For this reason, statistical analysis should be paired with visual inspection before final conclusions. The "Same Stats, Different Graphs" work by J. Matejka and G. Fitzmaurice gives a strong demonstration (See Figure 1.2).

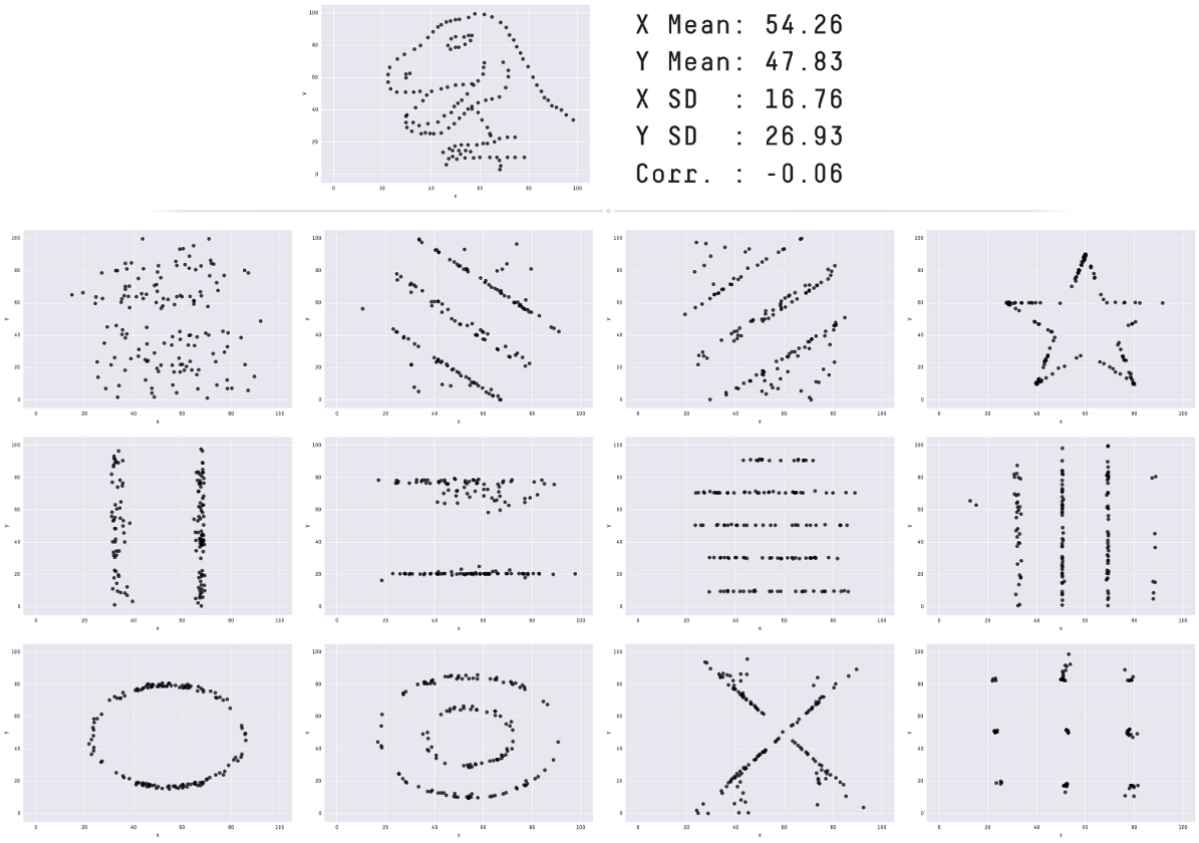

Figure 1.2: The Datasaurus Dozen: datasets with very different shapes but matching summary statistics (mean, standard deviation, and Pearson correlation) to two decimal places. This shows why visual inspection is essential in analysis and communication (Matejka and Fitzmaurice 2017).

1.4 Practical Workflow and Limits

Visualisation design follows an iterative workflow: **collect data**, **assess data quality**, **choose/design an idiom**, **evaluate and refine** (including clutter control, aesthetics, and effectiveness), and **communicate**. The steps are not linear; revisions are expected whenever new issues appear.

It also highlights practical limits. Very large datasets can exceed compute or rendering capacity; screen space is finite; and human perception is powerful but imperfect. In real projects, we often trade off between showing more data and preserving readability.

LIMITS NOTE
Three practical constraints:
• ***Computer limits*** *(especially interactive performance)*
• ***Human limits*** *(perceptual and cognitive)*
• ***Display limits*** *(finite pixels)*
Design always trades off data density against clutter.
Examples: Wikipedia: Misleading graph; ABC News: Read between the lines.

1.5 Analysis Framework: Four Levels and Three Questions

Once we know why visual inspection matters, we need a systematic method for design and critique. Munzner proposes four levels of design: (1) domain situation, (2) data/task abstraction, (3) visual encoding and interaction, and (4) algorithm (Munzner 2014). Across these levels, we repeatedly ask three linked questions: *why* is the task performed, *what* data is involved, and *how* should the visualisation encode and support interaction.

Figure 1.3 and Table 1.1 present complementary perspectives on the same relationship: *What* and *Why* are addressed primarily at the abstraction level, *How* at the idiom level, while algorithms realise the idiom within domain constraints. Used together, the figure conveys the conceptual nesting, and the table specifies the explicit mapping adopted in this unit.

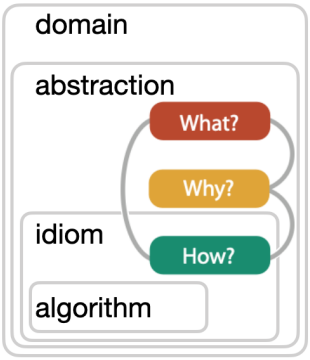

Figure 1.3: Relationship between the four levels (domain, abstraction, idiom, algorithm) and the three questions (What, Why, How) (Munzner 2014).

The 3 Questions	The 4 Levels (Nested Model)	Focus
(Context)	**1. Domain Situation**	Who are the users?
What?	**2. Data & Task Abstraction**	What data is used?
Why?	**2. Data & Task Abstraction**	What is the goal?
How?	**3. Idiom**	Visual design choices.
(Implementation)	**4. Algorithm**	Efficient computer code.

Table 1.1: Summary mapping between the Three Questions and the Four Levels (Nested Model) (Munzner 2014).

1) Domain situation

The first level identifies the domain context: who the users are, what domain-specific language they use, and which constraints shape their workflow. Without this grounding, later design choices may be technically correct but practically irrelevant. In a representative example, the domain is urban public transport planning and the user is a punctuality analyst.

2) Data and task abstraction (What and Why)

At this level, we translate domain language into abstract data and task language. In practice, this means clarifying *what* data structures are present and *why* users need to inspect them. [1]

[1] For example, train stations and tracks become nodes and links; the goal (why) may be understanding travel-time access to stops. Once these points are explicit, later encoding decisions become easier to justify and evaluate.

3) Visual encoding and interaction (How)

The third level turns abstraction into a visual idiom. At this stage, we choose marks, channels, layouts, and interactions that directly support the goals identified at the previous level.

4) Algorithm

The fourth level concerns implementation. Here, we focus on the algorithms and engineering choices needed to deliver the chosen idiom at useful speed and scale.

1.6 Data Abstraction (The "What")

With the framework in place, we now unpack the three guiding questions one by one. We start with *what*: the structure and meaning of the data.

Data abstraction describes data structure and meaning independently of domain details. It helps us choose suitable encodings and avoid misleading designs.

> **WHAT NOTE**
> *For more detail on **What**, see **Topic: Data Types**.*

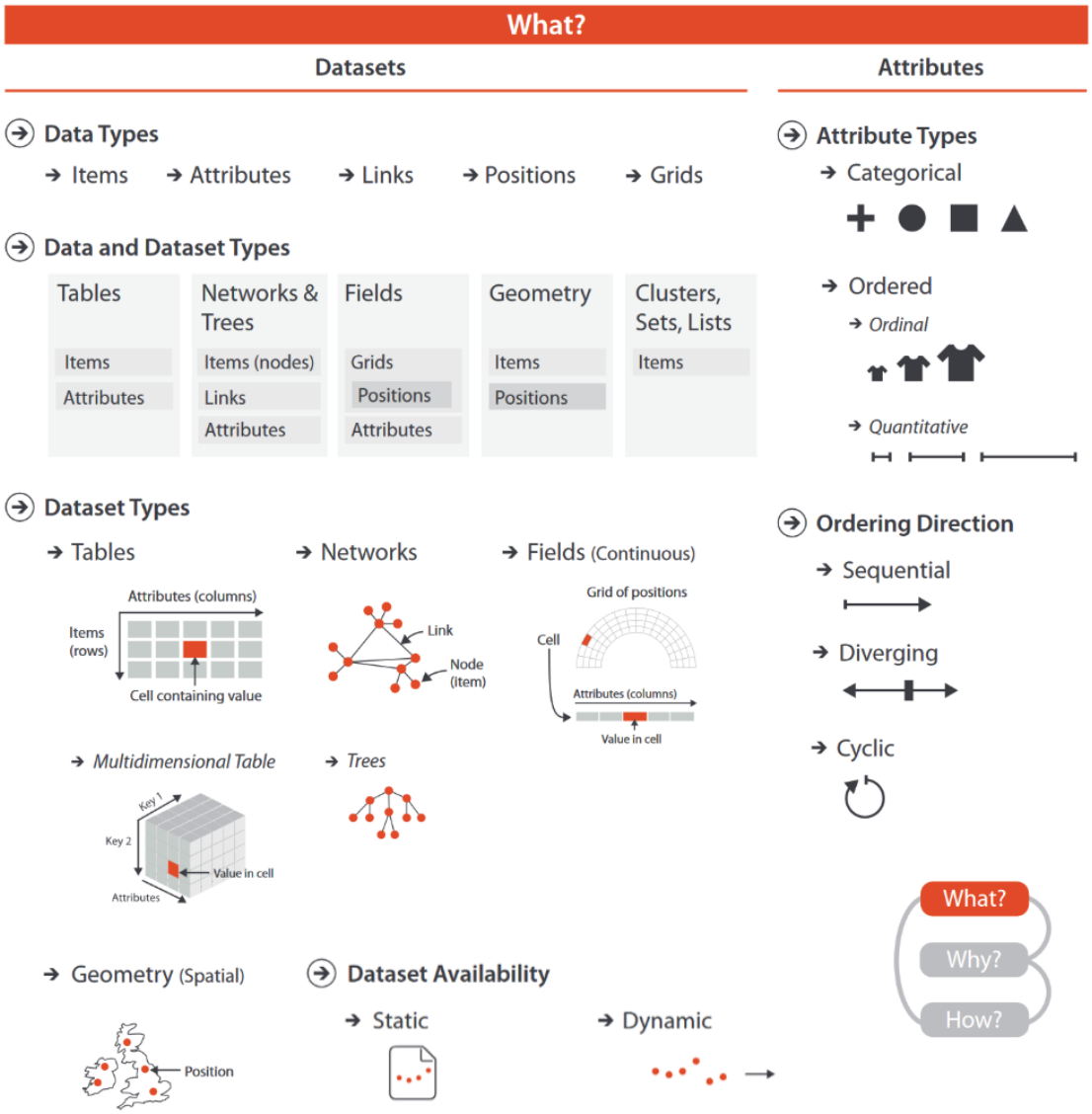

Figure 1.4: Munzner's *what* abstraction space: classify **dataset structure** first, then classify **attribute type and semantics**, before choosing any encoding (Munzner 2014).

Figure 1.4 should be read in two passes. First identify dataset structure (such as table, network, field, or geometry), because structure determines valid operations and layouts. Then classify attributes (categorical vs ordered; key vs value), because attribute semantics determine which channels can encode values correctly (Munzner 2014).

Data and dataset types

At the lowest level, we describe data using **items**, **attributes**, **links**, **positions**, and **grids**. Items are the entities we analyse, attributes are their properties, and links describe relationships between items. Positions and grids become important when the data is spatial or sampled from continuous space.

These building blocks combine into familiar dataset forms. **Tables** organise rows and columns; **networks (graphs)** organise nodes and links; **fields** [2] sample values across space, often as scalar, vector, or tensor quantities; and **geometry** carries explicit shape and location. This classification helps prevent mismatches between data structure and chart form.

[2] Observed samples from an underlying continuous phenomenon.

A key operational difference is that tables can usually be reordered (for example, sort rows) without changing meaning, while spatial fields cannot be arbitrarily reordered because position is part of the data semantics.

Common Vocabulary
- **Network = graph**
- **Node = vertex**
- **Link = edge**

Attribute types

Attribute meaning is the second part of abstraction. **Categorical (nominal)** attributes encode identity without order, while **ordered** attributes encode rank or magnitude. Ordered attributes may be *ordinal*, where rank matters but distance does not, or *quantitative*, where arithmetic comparisons are valid. Direction then refines interpretation: values can be *sequential*, *diverging*, or *cyclic* [3].

[3] Typical examples: beach visits in a year are *sequential*, elevation above/below sea level is *diverging*, and weekday or month patterns are *cyclic*.

Semantics. Finally, we separate **keys** and **values**. Keys identify items and support lookup, while values store measured or derived quantities to compare.

1.7 Task Abstraction (The "Why")

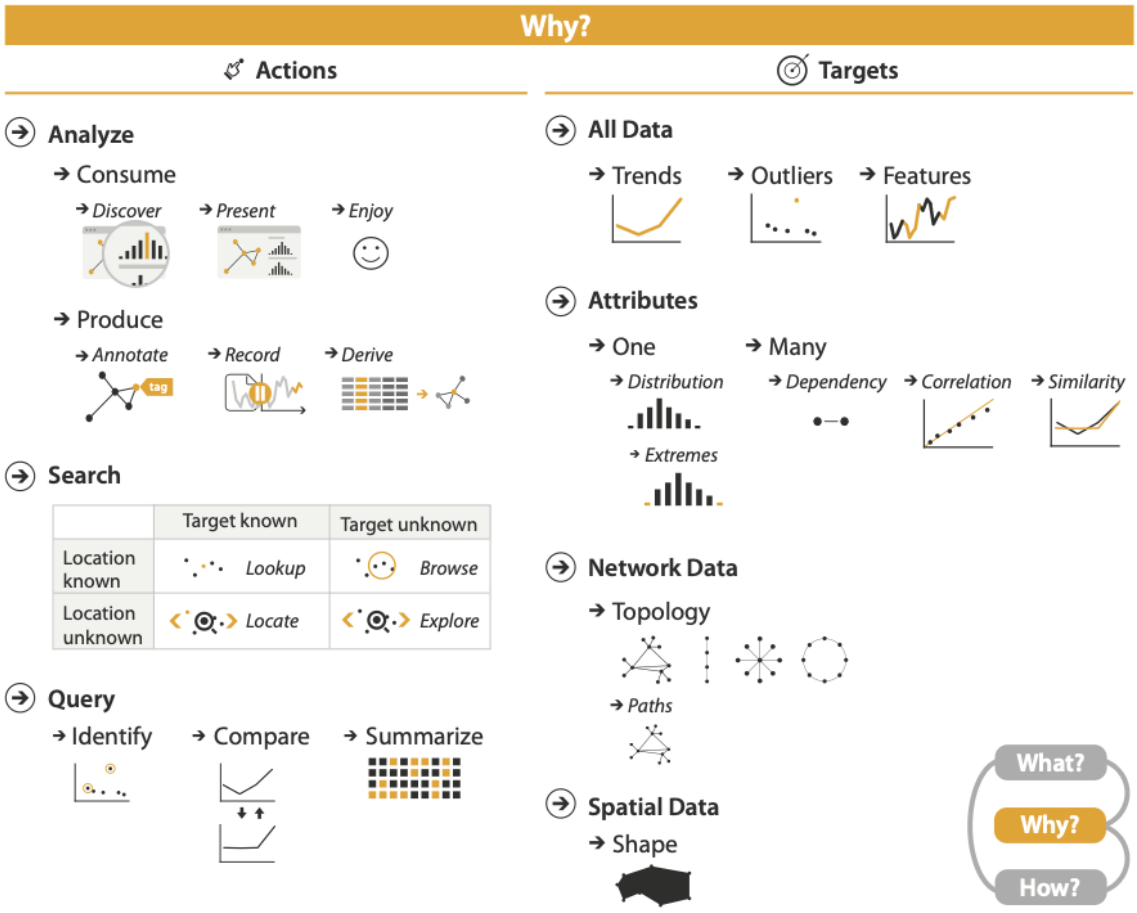

Figure 1.5: Munzner's *why* abstraction space: tasks are specified as **actions** applied to **targets** (Munzner 2014).

In this unit, *why* follows Munzner's task abstraction: goals are specified as **actions** applied to **targets**. Typical goals include identifying **correlation**, comparing **similarity**, understanding **distribution**, detecting **trends over time**, finding **outliers**, and recognising notable **features**.

In this **actions-and-targets** view, actions include **analyse** (consume, produce), **search**, and **query**; targets include all data, subsets, or individual attributes/relationships.

Figure 1.5 is read by combining one action family with one target type. On the action side, analysts may **analyse** (consume/produce), **search** (lookup, locate, browse, explore), or **query** (identify, compare, summarise). On the target side, tasks may concern all data, attribute relationships, network structure, or spatial structure. A specific task is therefore an action-target pair, such as **compare** + **attributes** or **explore** + **all data** (Munzner 2014).

In the **consume** branch, common intents are **discover** and **enjoy**; in the **produce** branch, common actions are **annotate**, **record**, **derive**, and

> **NOTE**
> *This section starts with Munzner's **action-target Why** model (Figure 1.5). For extended explanatory **Why** design, see **Topic: How to Tell a Story with Data Visualisation**.*

present. **Derive is especially important**: task-relevant transformed attributes are often easier to interpret than raw ones.

From Action-Target to Function–Form

We map Munzner's action-target *why* space to a **function** continuum: **Exploration** → **Explanation**.

In Munzner's full *why* taxonomy, **exploration** aligns mainly with **Analyze** → **Consume** → **Discover**, plus substantial **Search** (especially target/location unknown cases such as browse and explore) and **Query** (identify, compare, summarise). Exploration often includes **Produce** actions such as **Derive** and **Annotate** as intermediate analysis artefacts. **Explanation** aligns mainly with **Present**: communicating conclusions to an audience with clearer targets and more controlled paths.

In this unit, this mapping is a deliberate simplification of Munzner's richer action-target space. It clarifies scope: we retain the core *why* logic, but treat it as a progression from open-ended analysis to directed communication.

After discussing **function** (*why*) on the x-axis, we turn to **form** on the y-axis: **static** ↔ **interactive**. This is the *how* side of Figure 1.6. The two axes are related in practice but conceptually distinct.

SOURCES
Form vs Function
Schwabish 2021
Better Data Visualizations: A Guide for Scholars, Researchers, and Wonks

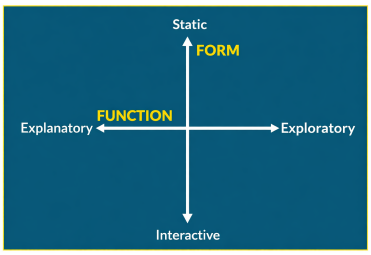

Figure 1.6: Function–Form Quadrant of Data Visualisation: **x-axis** is **function** (exploratory–explanatory, *why*); **y-axis** is **form** (static–interactive, mainly *how*).

1.8 Marks and Channels (The "How")

With *what* and *why* defined, we can now decide *how* to encode the data. Visual encoding maps abstract data to perceivable form. Good encodings make important patterns easy to see and hard to misread.

In this unit, **marks and channels** correspond roughly to the **static** part of Munzner's *how* space, mainly the **Encode** branch (map/express). The full *how* scope is broader: it also includes the **interactive** parts—**Manipulate** (interaction changes such as select, navigate, change), **Facet** (multi-view composition such as juxtapose, partition, superimpose), **Reduce** (filtering and aggregation), and **Embed** (focus+context). These interactive *how* components are introduced for context, but detailed mastery is **not required** in this unit.

HOW NOTE
*For more detail on **How**, see **Topic: Marks and Channels**.*

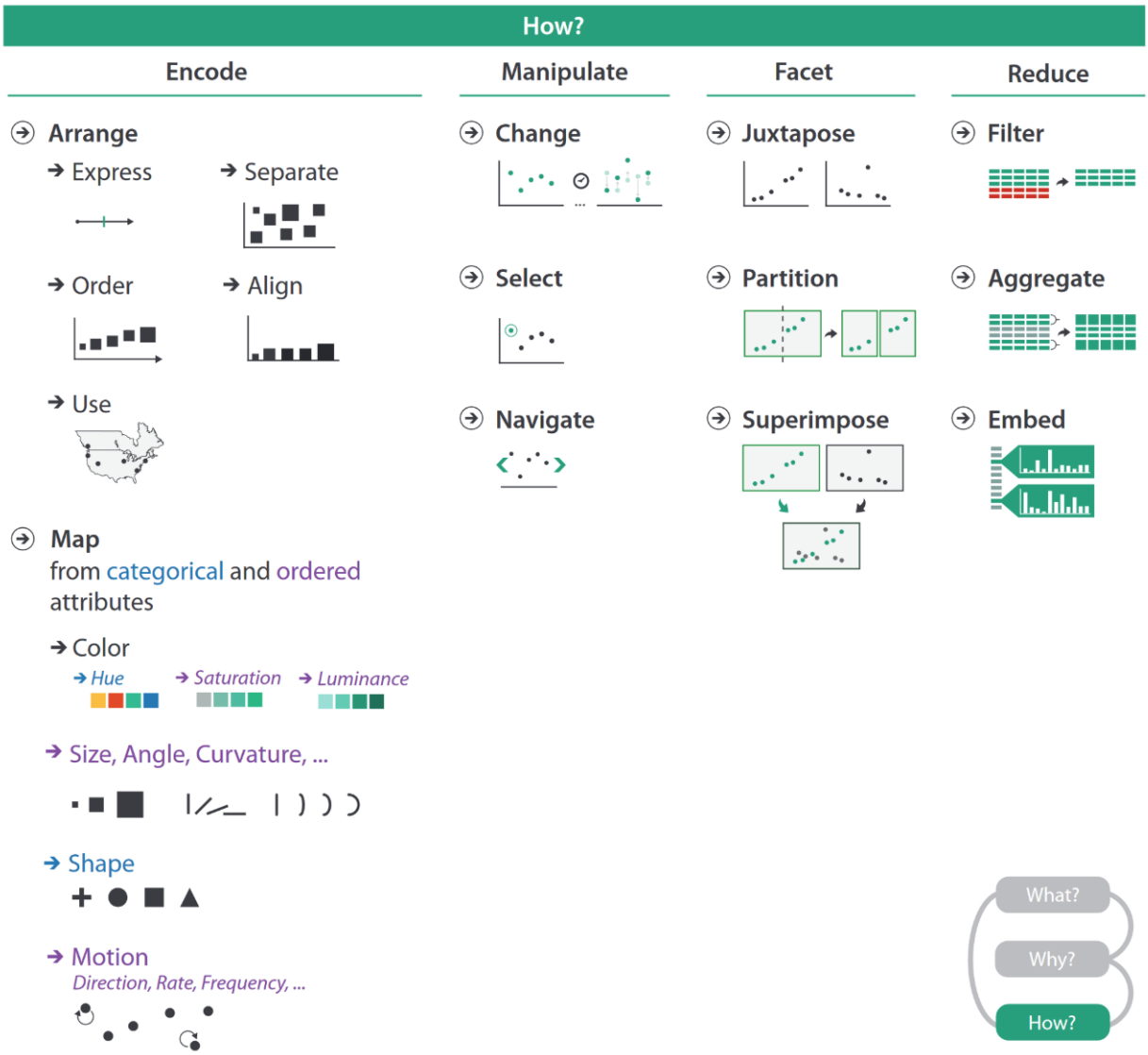

Figure 1.7: Munzner's *how* design space: **Encode**, **Manipulate**, **Facet**, and **Reduce**. In this unit, required mastery is the static **Encode** branch, with other branches introduced as context (Munzner 2014).

Figure 1.7 clarifies that *how* is broader than a chart type. **Encode** selects marks and channels; **Manipulate** supports interaction changes; **Facet** organises multi-view composition; and **Reduce** controls scale through filtering or aggregation. In this topic, we go deepest on **Encode** so that encoding decisions remain consistent with the previously defined *what* and *why* (Munzner 2014).

Encode

Marks are geometric primitives that represent items and links, and the most common mark types are *points* (0D), *lines* (1D), and *areas* (2D). In practice, lines often communicate *connection* and areas often communicate *containment*. A key reminder is that a mark specifies geometry, not appearance. **Channels** control appearance and carry data meaning

through properties such as position, size, shape, orientation, colour, and motion.

For colour, the three channels are **hue** (which pure colour), **luminance/value** (light vs dark), and **saturation** (gray vs pure colour).

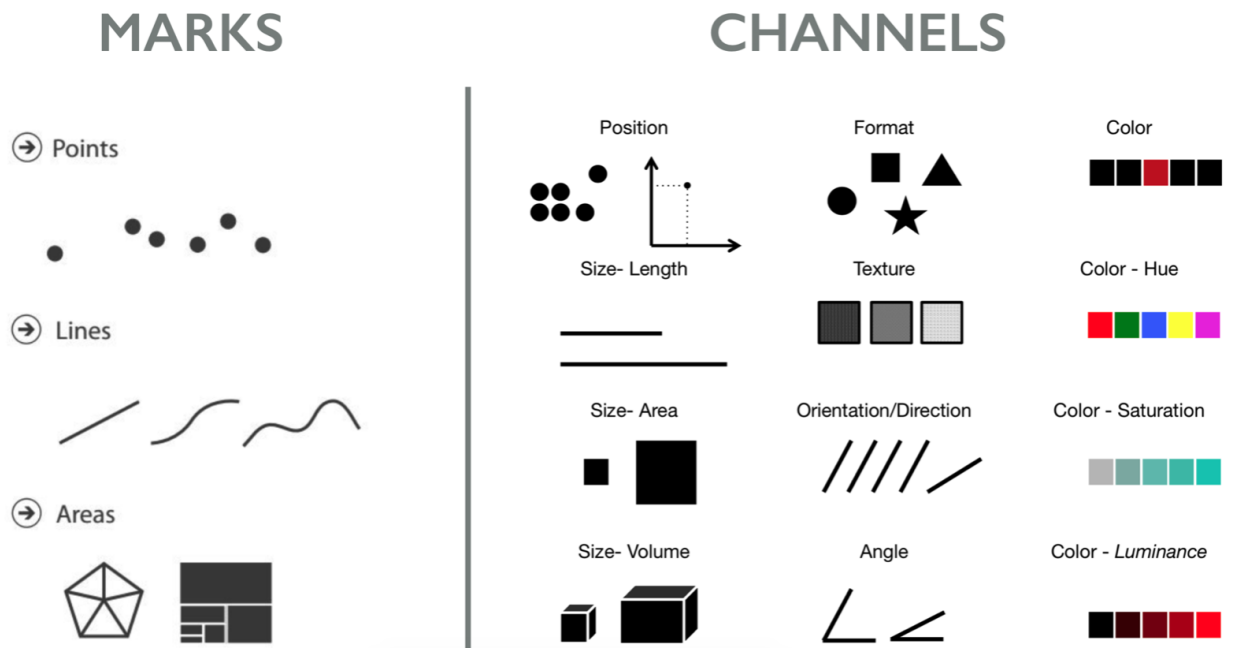

Figure 1.8: Static encoding focus for this unit: marks and channels matrix showing which channels are available for points, lines, and areas, and how suitability differs for ordered versus categorical attributes (Munzner 2014).

Figure 1.8 is the practical lookup used in this topic: first choose a mark family, then choose channels that match the attribute type and task.

Redundant and Multivariate Encoding

Redundant encoding means using multiple channels to represent the same attribute (e.g., both colour and shape for category), which can improve robustness and readability, especially if one channel is hard to perceive. **Multivariate encoding** means using different channels to encode different attributes, allowing more information to be shown at once. Both strategies should be used deliberately, based on task needs and perceptual limits.

CORE PRINCIPLES
Two principles guide these decisions. ***Expressiveness*** *means the encoding should represent all and only the intended information, while* ***effectiveness*** *means the most important attributes should be mapped to the channels people can judge most accurately.*

Channel effectiveness rankings

For **ordered data** (“how much?”), effectiveness generally follows this pattern: **position on a common scale** is strongest, followed by length, then angle/orientation and area, while volume and colour-based magnitude judgements (luminance/saturation) are usually weakest for precise reading. For **categorical data** (“what/where?”), spatial region and colour hue are usually strong choices, with shape also useful for identity.

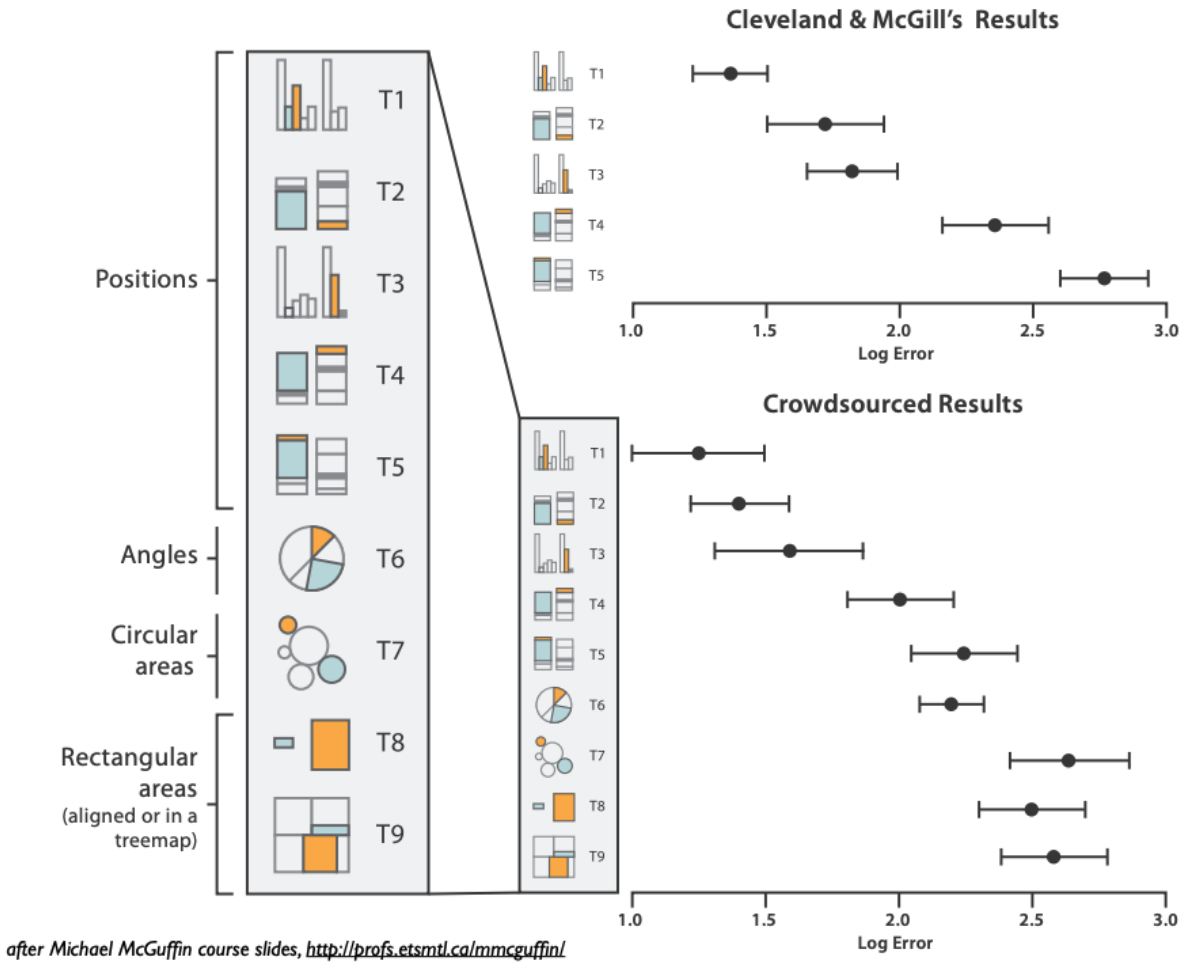

Figure 1.9: Graphical perception results. Left: ranked task types by channel. Right: log error in Cleveland & McGill and in Heer & Bostock's crowdsourced replication (Heer and Bostock 2010).

Figure 1.9 grounds the ranking list empirically: channels tied to position and length yield lower estimation error than area, volume, and colour-based magnitude judgements.

SOURCES
Heer and Bostock 2010
"Crowdsourcing Graphical Perception: Using Mechanical Turk to Assess Visualization Design"

1.9 Example: Gapminder Bubble Chart

To make the framework concrete, we use the Gapminder bubble chart (life expectancy vs GDP per capita over time) as a worked example (Gapminder Foundation 2026).

SOURCES
The Gapminder Example
Gapminder Foundation 2026
Gapminder Tools
Open source link

What: The dataset contains country-level observations of life expectancy, GDP per capita, population, world region, and year. The main **items** are countries observed over time. Key **attributes** are GDP per capita and life expectancy (**quantitative**), population (**quantitative**), world region (**categorical**), and year (**ordered** temporal). This makes the dataset suitable for comparing cross-country differences and temporal change.

Why: The main goal is to understand long-run development patterns across countries. In **actions-and-targets** terms, viewers **query** to com-

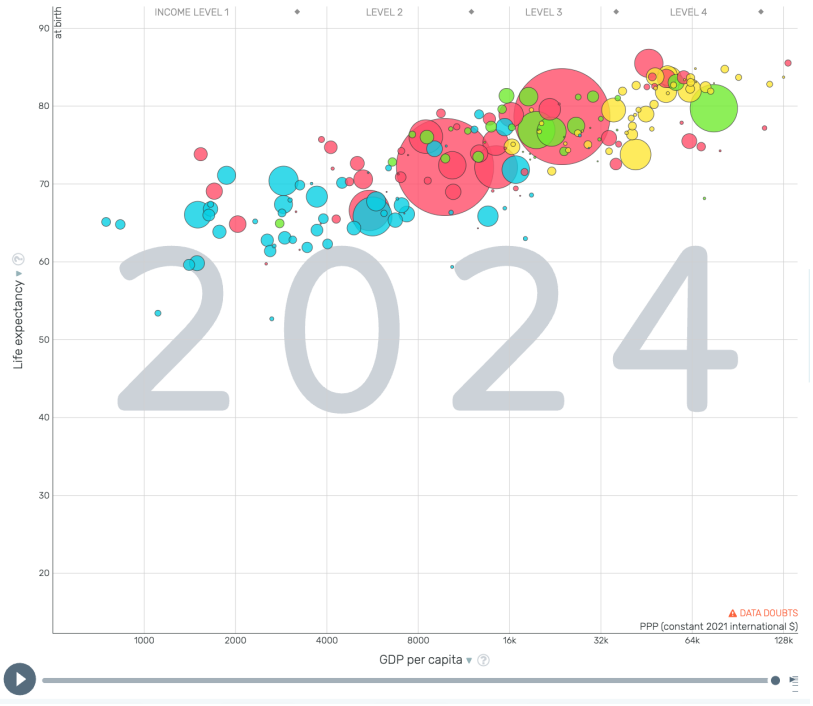

Figure 1.10: Gapminder bubble chart example used to explain *what–why–how* (Gapminder Foundation 2026).

pare countries, **search** for outliers, and **analyse** trajectories through time. In **function** terms, the same chart supports **Exploration** (open-ended inspection and hypothesis building) and **Explanation** (communicating long-run development patterns to a broader audience).

How: The **idiom** is an animated bubble scatterplot with circle **marks**. Primary **channels** are x-position for GDP per capita, y-position for life expectancy, **size** for population, and **colour hue** for world region; time is shown with motion along the timeline. In **form** terms, interaction (play/pause, scrubbing, hover/filter) supports exploratory reading, while static snapshots can support explanatory communication.

This example shows the full design chain in order: define *what* data exists, specify *why* tasks and **function**, then choose *how* through **form**, **marks**, and **channels**. Using this sequence helps keep visual decisions aligned with analytical goals instead of chart style alone.

Bibliography

Anscombe, Francis J. (1973). “Graphs in Statistical Analysis”. In: *The American Statistician* 27.1, pp. 17–21.

Gapminder Foundation (2026). *Gapminder Tools*. URL: https://www.gapminder.org/tools/#$chart-type=bubbles&url=v2 (visited on 02/27/2026).

Heer, Jeffrey and Michael Bostock (2010). “Crowdsourcing Graphical Perception: Using Mechanical Turk to Assess Visualization Design”. In: *Proceedings of the ACM Conference on Human Factors in Computing Systems (CHI)*. ACM, pp. 203–212.

Kirk, Andy (2019). *Data Visualisation: A Handbook for Data Driven Design*. 2nd ed. SAGE Publications.

Matejka, Justin and George Fitzmaurice (2017). “Same Stats, Different Graphs: Generating Datasets with Varied Appearance and Identical Statistics through Simulated Annealing”. In: *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. ACM, pp. 1290–1294. DOI: 10.1145/3025453.3025912. URL: https://www.research.autodesk.com/publications/same-stats-different-graphs/.

Munzner, Tamara (2014). *Visualization Analysis and Design*. AK Peters Visualization Series. CRC Press.

Schwabish, Jonathan A. (2021). *Better Data Visualizations: A Guide for Scholars, Researchers, and Wonks*. New York: Columbia University Press.